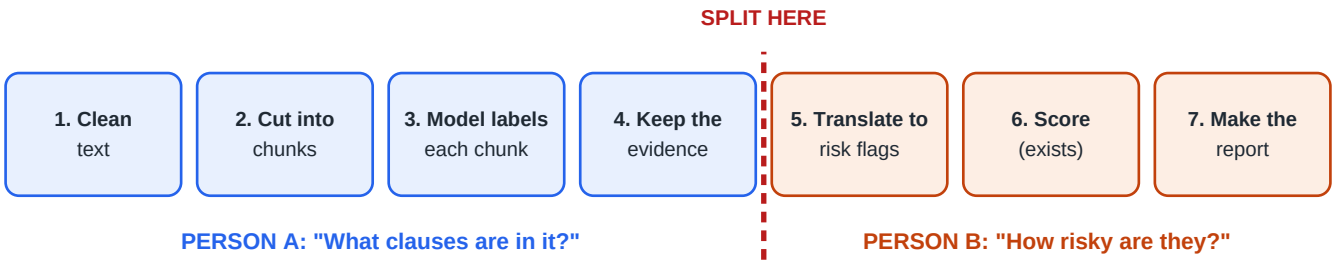
Building the End-to-End Pipeline

How the November milestone works, and how two people can build it at the same time

The November milestone asks us to "assemble the end-to-end pipeline": one program where a raw contract goes in and a risk report comes out, with no manual steps in between. Most pieces already exist separately. Our job is to connect them, and to fill in the one piece that is missing (stage 5).



Final output for every contract: clause type -> evidence (the actual text) -> risk level -> reason -> overall contract score.

The pipeline, following one contract

Imagine a 30-page supply agreement between Acme and a vendor.

1 — Clean the text
Remove junk like page numbers, repeated headers and extra spaces, without changing any legal wording.

2 — Cut it into chunks
The AI model can only read about a page at a time (512 tokens, roughly 350-400 words). Cut the contract into page-sized pieces that overlap a little (64 tokens), so a clause on a cut line still appears whole in at least one piece. The 30-page contract becomes about 40 chunks.

3 — The model labels each chunk
This is the AI the team trained in October. For each chunk it answers: "which of the 41 clause types does this piece contain?" For example, chunk 17 = "Cap On Liability" (91% confident), chunk 22 = "Termination For Convenience" (84%), chunk 30 = nothing. **The model only says what kind of clause it found. It has no idea whether the clause is good or bad.**

4 — Keep the evidence
Save the actual chunk text behind every label, so a human can see why the model said so.

5 — Translate clause types into risk flags (the missing piece)
The risk scorer does not use clause types. It uses 8 yes/no risk questions, such as "is liability uncapped?" or "is indemnification missing?" Something has to translate: chunk 17 is a liability clause; does it name a dollar cap? If not, *uncapped_liability* = present. No chunk anywhere was labelled Indemnification, so *missing_indemnification* = present. If we can't tell, mark it *unknown* and a human reviews it.

6 — Score it (already built)
Feed the flags into the existing *risk_scoring* code. It returns a 0-100 score and Low/Medium/High per clause, and gives the contract the score of its worst clause. We just call it.

7 **Make the report**
Output something a reviewer can read, for example:

Clause	Evidence	Risk	Why
Liability	"Vendor shall be liable for all damages..."	High	No cap on liability
Termination	"Acme may terminate at any time..."	Medium	One-sided termination
Contract score		**High**	

Who does what

	Person A: "What's in it?"	Person B: "How risky is it?"
Stages	1-4	5-7
Work	Move the cleaning, chunking and model code out of the October Colab notebooks into regular Python files. Save the trained model so it can be reloaded. Run it on the test contracts and write the hand-off file.	Write the translation rules (stage 5). Call the existing scorer. Build the report. Collect the advisor's 1-5 ratings and compare them with ours.
Needs	A GPU (Colab Pro or Kaggle). Comfort with the October training notebooks.	Just a laptop. Some judgement about contract language.
Hand-off	**Produces** a file listing each contract's chunks and labels	**Consumes** that file
Good fit if	You know the model and training notebooks better.	You are still learning the pipeline: the scorer already exists and everything can be tested on a laptop.

Why this lets us work at the same time

The only thing the two halves share is the **hand-off file**: the list of chunks and their labels. On day one, we write a fake version by hand together (one made-up contract with a few made-up labels). Person B builds all of stages 5-7 against the fake file while Person A builds the real thing. When Person A's real file is ready, we swap it in.

Example hand-off file (detections.json)

```
{
  "contract_id": "AcmeCorp_Supply_Agreement",
  "split": "test",
  "chunks_scanned": 42,
  "detections": [
    {
      "category": "Cap On Liability",
      "chunk_id": 17,
      "text": "...the full chunk text...",
      "probability": 0.91,
      "threshold": 0.5
    }
  ]
}
```

Why "chunks_scanned" matters: to say "this contract has no indemnification clause", Person B must know the model checked the *whole* contract. The scorer refuses to call something missing based on a single chunk.

Plan

When	What
Day 1, together	Agree on the hand-off file format. Write the fake example file. Create two branches (e.g. pipeline-model and pipeline-scoring) so we never edit the same files.
Working separately	Person A: stages 1-4, producing the real hand-off file. Person B: stages 5-7, tested against the fake file. Person B also sends the advisor the clause packet to rate.
Join up	Run Person A's real file through Person B's code. Expect small format fixes.
Together, to finish	One command that runs everything (e.g. python -m pipeline run). Check results against the advisor's ratings (ranking correlation, how often Low/Medium/High match, and how many truly high-risk contracts we caught). Update the README.

Open question for the advisor

The risk scorer only lets us tune its settings on train/validation contracts, never test contracts, so we are not "grading our own homework". But the advisor's FAQ answer said to pull the 20-30 example clauses "from the test set". We should confirm which set the advisor wants to rate before sending the packet.